

Session 12.

1. Create a simple React component called `TrendingSongs` that uses `fetch()` to get data from `https://jsonplaceholder.typicode.com/posts` and displays the first 3 titles. If the API call fails, show a message 'Error loading data' instead of the list.
2. Modify your `TrendingSongs` component to include a 'Reload' button that retries the API call when clicked, and keeps showing the error message if the fetch still fails.

Hint: Use `try/catch` inside an `async` function and update state to trigger a re-render.
3. Build a React component called `IPLScores` that fetches and displays dummy cricket scores from a mock API endpoint (use `https://jsonplaceholder.typicode.com/users`). If the response status is not 200, throw an error and display 'Error loading scores'.
4. Given the following buggy code in a React `useEffect` hook:

```
hook: useEffect(() => {  
  fetch('https://jsonplaceholder.typicode.com/invalidurl').then(res =>  
    res.json()).then(data => setData(data)).catch(err => setError(true));  
});
```


Fix the code so that it correctly handles both network errors and non-200 HTTP status codes, showing an error message if either occurs.

Hint: Use `try/catch` and check `response.ok`.

Answer: -

```
import React, { useEffect, useState } from "react";

function TrendingSongs() {
  const [songs, setSongs] = useState([]);
  const [error, setError] = useState(false);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((response) => response.json())
      .then((data) => {
        setSongs(data.slice(0, 3));
      })
      .catch(() => {
```

```

        setError(true);
    });
}, []);

if (error) {
    return <h2>Error loading data</h2>;
}

return (
    <div>
        <h2>Trending Songs</h2>

        {songs.map((song) => (
            <p key={song.id}>{song.title}</p>
        ))}
    </div>
);
}

export default TrendingSongs;

```

```

import React, { useEffect, useState } from "react";

function TrendingSongs() {
    const [songs, setSongs] = useState([]);
    const [error, setError] = useState(false);

    const fetchSongs = async () => {
        try {
            setError(false);

            const response = await fetch(
                "https://jsonplaceholder.typicode.com/posts"
            );

            if (!response.ok) {
                throw new Error("Failed to fetch data");
            }

            const data = await response.json();
            setSongs(data.slice(0, 3));
        } catch (err) {
            setError(true);
            setSongs([]);
        }
    };
}

```

```

useEffect(() => {
  fetchSongs();
}, []);

return (
  <div>
    <h2>Trending Songs</h2>

    <button onClick={fetchSongs}>Reload</button>

    {error ? (
      <p>Error loading data</p>
    ) : (
      songs.map((song) => (
        <p key={song.id}>{song.title}</p>
      ))
    )}
  </div>
);
}

export default TrendingSongs;

```

```

import React, { useEffect, useState } from "react";

function IPLScores() {
  const [scores, setScores] = useState([]);
  const [error, setError] = useState(false);

  const fetchScores = async () => {
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/users"
      );

      if (response.status !== 200) {
        throw new Error("Invalid response");
      }

      const data = await response.json();
      setScores(data);
    } catch (err) {
      setError(true);
    }
  };

  useEffect(() => {

```

```

    fetchScores();
  }, []);

  if (error) {
    return <h2>Error loading scores</h2>;
  }

  return (
    <div>
      <h2>IPL Scores</h2>

      {scores.map((team) => (
        <div key={team.id}>
          <p>
            <strong>{team.name}</strong> - {Math.floor(Math.random() * 250)}/
            {Math.floor(Math.random() * 10)}
          </p>
        </div>
      ))}
    </div>
  );
}

export default IPLScores;

```

```

useEffect(() => {
  const fetchData = async () => {
    try {
      const response = await fetch(
        "https://jsonplaceholder.typicode.com/invalidurl"
      );

      if (!response.ok) {
        throw new Error(`HTTP Error: ${response.status}`);
      }

      const data = await response.json();
      setData(data);
    } catch (error) {
      setError(true);
    }
  };

  fetchData();
}, []);

```